EPFL - Computer Science

CS-550 Formal Verification

Lecture 02 — October 18, 2026

Author: Jonathan Dutoit

Professor: Viktor Kuncak

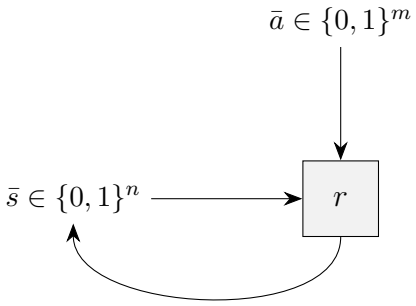
Finite Systems Expressed with Formulas

Symbolic model checking: Explicitly storing all reachable states is often too expensive. Instead, represent state sets compactly using formulas or data structures. Two important techniques for finite-state systems are **bounded model checking**, which uses **SAT solvers** to . search for errors within a fixed bound, and **BDD**-based reachability checking, which computes reachable-state sets symbolically until a fixed point is reached.

Boolean Functions: Encoding FTS (Finite Transition Systems)

Consider a deterministic finite-state transition system: $M = (S, I, r, A)$. If we pick $n \geq log_2|S|$ and $m \geq log_2|A|$, we can represent the finite-state transition system using boolean functions:

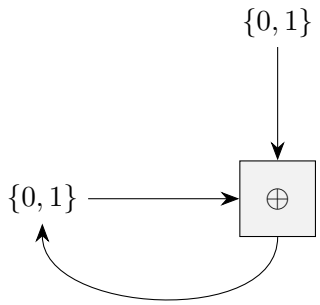
- Each element of S as $\bar{s} \in \{0,1\}^n$, so $S = \{0,1\}^n$
- Each element of A as $\bar{a} \in \{0,1\}^n$, so $A = \{0,1\}^n$
- Initial states $I \subseteq S$ by the characteristic function $\{0,1\}^n \to \{0,1\}$
- Deterministic transition relation $r \subseteq S \times X \times S$ as a function $(S \times A) \to S$, that is, $\{0,1\}^n \times \{0,1\}^m \to \{0,1\}^n$

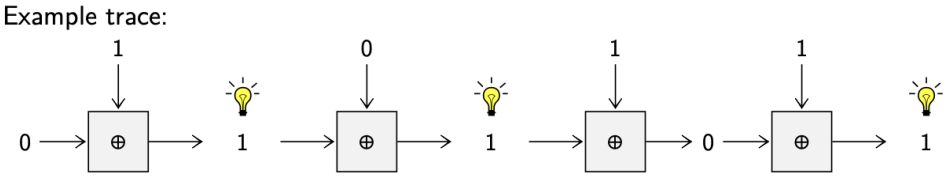


(For non-deterministic systems, we represent r as $(S \times A \times S \to \{0,1\})$

Example: Blinking Lights:

$S = \{0,1\}$ (1 = "light on") $A = \{0,1\}$ (1 = "toggle light") $I(s) = (s = 0)$ $r(s,a) = s \oplus a$





Boolean Function Representation: Table

Let r: $\{0,1\}^n \times \{0,1\}^m \to \{0,1\}^n$. The table list each of the $\{0,1\}^{n+m}$ inputs and specify the result. When $s, a \in \{0,1\}^4$ and represent addition of 4-bit non-negative integers modulo 16:

s	a	$r(s,a)$
0000	0000	0000
0000	0001	0001
0000	0010	0010
	...	
0001	0000	0001
0001	0001	0010
	...	
1111	1111	1110

$2^{4+4} = 256$ columns

In general, we cannot do better than truth table (there are that many different functions), but truth table always has bad representation size, even for functions like $+_4$.

What happens in practice: in verification we don't deal with random systems. We care about meaningful systems that have a more concise representation and are computable functions.

Boolean Function Representation: Formulas

Let $r \subseteq \{0,1\}^n \times \{0,1\}^m \times \{0,1\}^n$. **Definition.** Without loss of generality, we can always represent r by a propositional formula in disjunctive normal form:

$$\bigvee_{((v_1,...,v_n),(u_1,...,u_m),(v'_1,...,v'_n))\in r} \left(\bigwedge_{1\le i\le n} s_i^{v_i} \wedge \bigwedge_{1\le 1\le m} a_i^{u_i} \wedge \bigwedge_{1\le 1\le n} (s'_i)^{v'_i} \right).$$

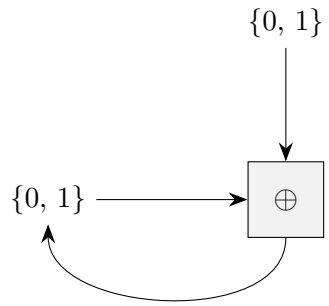
Where we represent the condition

$$((s_1,...,s_n),(a_1,...,a_m),(s'_1,...,s'_n)) \in r.$$

by writing a propositional formula with variable $s_1,...,s_n$, $a_1,...,a_m$, $s'_1,...,s'_n$ that is *true* precisely when the tuple belongs to r, and if p is a propositional variable and $v \in \{0,1\}$ then we define p^v by $p^1 = p$ and $p^0 = \neg p$

Example: Blinking Lights

$S = \{0,1\}$ (1 = "light on") $A = \{0,1\}$ (1 = "toggle light") $I(s) = (s = 0)$ $r(s,a) = s \oplus a$



Formula R represents condition $(s, a, s') \in r$:

r $(relation)$	R $(formula)$
$\{(0,0,0),$	$(\neg s \wedge \neg a \wedge \neg s') \vee$
$(0,1,1),$	$(\neg s \wedge a \wedge s') \vee$
$(1,0,1),$	$(s \wedge \neg a \wedge s') \vee$
$(1,1,0)\}$	$(s \wedge a \wedge \neg s')$

Propositional (Boolean) Logic

Propositional logic is a language for representing Boolean functions $f : \{0,1\}^n \to \{0,1\}$. Grammar of formulas:

$$P ::= x \,|\, 0 \,|\, 1 \,|\, P \wedge P \,|\, \neg P \,|\, P \vee P \,|\, P \oplus P \,|\, P \to P \,|\, P \leftrightarrow P.$$

where x denotes variables (identifiers). Corresponding Scala trees:

```
sealed abstract class Expr
case class Var(id: Identifier) extends Expr
case class BooleanLiteral(b: Boolean) extends Expr
case class And(e1: Expr, e2: Expr) extends Expr
case class Or(e1: Expr, e2: Expr) extends Expr
case class Not(e: Expr) extends Expr
```

Definition. An environment e is a partial map from propositional variables to $\{0,1\}$. For a vector of n boolean variables $\bar{p} = (p_1, ..., p_n)$ and $\bar{v} = (v_1, ..., v_n) \in \{0,1\}^n$, we denote $[\bar{p} \mapsto \bar{v}]$ the environment e given by $e(pi) = v_i$ for $i \leq i \leq n$.

We write $e \models F$, and define $[\![F]\!]_e = 1$, to denote that F is true in environment e, otherwise we define $[\![F]\!]_e = 0$.

Example: Let $e = \{a, 1), (b, 1), (c, 0)\}$ and F be $a \wedge (\neg b \vee c)$. Then:

$$[\![a \wedge (\neg b \vee c)]\!]_e = e(a) \wedge (\neg e(b) \vee e(c)) = 1 \wedge (\neg 1 \vee 0) = 0.$$

The general definition is recursive for assigning truth values to formulas:

$$\begin{aligned} &[\![x]\!]_e = e(x) \\ &[\![0]\!]_e = 0 \\ &[\![1]\!]_e = 1 \\ &[\![F_1 \wedge F_2]\!]_e = [\![F_1]\!]_e \wedge [\![F_2]\!]_e \\ &[\![\neg F_1]\!]_e = \neg [\![F_1]\!]_e \end{aligned} \quad .$$

Satisfiability Problem

Definition. Formula F is *satisfiable*, iff there **exists** e such that $[\![F]\!]_e = 1$.
Otherwise we call F *unsatisfiable*: when there does not exist e such that $[\![F]\!]_e = 1$, that is, for all e, $[\![F]\!]_e = 0$.

SAT is a problem: given a propositional formula. determine whether it is satisfiable. The problem is **decidable** because given F we can compute its variables *FV(F)* and it suffices to look at the 2^n environments for n = *FV(F)*. The problem is NP-complete, but useful heuristics exist.

Example: Let F be a $a \wedge (\neg b \vee c)$. Then F is satisfiable, with e.g. $e = \{(a, 1), (b, 0), (c, 0)\}$. Its negation of $\neg F = \neg a \vee (b \wedge \neg c)$, is also satisfiable, with e.g. $e = \{(a, 0), (b, 0), (c, 0)\}$.

A SAT solver is a program that, given boolean formula F, either:

- returns **sat**, and, optionally, returns one environment e such that $[\![F]\!]_e = 1$, or
- returns **unsat** and, optionally, returns a **proof** that no satisfying assignment exists. Either the program tries every environment possibility, or uses some observations to evaluate a more concise equivalent formula.

Syntax vs Semantics

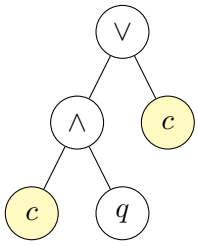
- **Syntax (Form & Structure):** The symbols and mechanical rules for writing formulas.
  - $(p \wedge q)$ is a valid formula; substitution $(F[p := q])$ is pure text replacement.
- **Semantics (Meaning & Truth):** What those symbols actually evaluate to under an interpretation.
  - Assigning p = True and q = False, or checking if an environment e satisfies a formula $(e \models F)$.

Substitutions

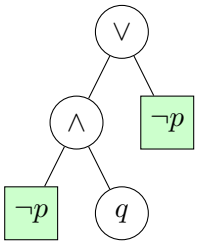
Let F, G be propositional formulas and c a propositional variable.
Let $F[c := G]$ denote the result of replacing in F each occurrence of c by G recursively.

Example $F[c := \neg p]$

Original Formula $F = (c \wedge q) \vee c$



After Substitution $F[c := \neg p]$



The general definition is:

$$\begin{aligned} c[c := G] &= G \\ (F_1 \wedge F_2)[c := G] &= F_1[c := G] \wedge F_2[c := G] \\ (F_1 \vee F_2)[c := G] &= F_1[c := G] \vee F_2[c := G] \\ (\neg F_1)[c := G] &= \neg(F_1[c := G]) \end{aligned}.$$

We also generalize to simultaneous replacement of many variables, $F[\bar{c} := \bar{G}]$.
Then following formulas are equivalent (have same truth for all free variables):

- $F[c := G]$
- $\exists c.((c = G) \wedge F)$ *satisfied only if F evaluates to true when c is replaced inside G*
  - The existential quantifier $(\exists c)$ looks across all possible values for c.
  - The subformula requires **both** $c = G$ and F to be true
  - Because of $c = G$, the only value of c that can possibly satisfy this condition is whatever G evaluates to

  - When c takes that specific value, checking if F is true is the exact same thing as taking F and replacing every c with G.

- $\forall c.((c = G) \to F)$ *The only value of c that actually tests F is when $C = G$. This forces F to hold precisely where c has been replaced by G*
  - If c **is not** equal to G, then the premise $(c = G)$ is **false**. In logic, a false premise makes an implication **automatically** (vacuously) true. So all other values of c get a free pass.
  - The only time the premise is actually tested is when c **does** equal G.
  - When $c = G$, the implication forces F to be true.

Note: free variables are the variables occurring in the formula minus quantified ones (c).

Free Variables for Quantified Boolean Formulas

Quantified boolean formulas (QBF) are build from propositional variables and constants 0, 1 using $\wedge, \vee, \neg, \to, \leftrightarrow, \exists, \forall$ (We also write $=$ for $\leftrightarrow$).
A boolean formula is a QBF without quantifiers $\forall, \exists$.
Definition of free variables of a formula:

$$\begin{aligned} FV(v) &= \{v\} \text{ when v is a propositional variable} \\ FV(F_1 \wedge F_2) &= FV(F_1) \cup FV(F_2) \\ FV(F_1 \vee F_2) &= FV(F_1) \cup FV(F_2) \\ FV(F_1 \to F_2) &= FV(F_1) \cup FV(F_2) \\ FV(\neg F_1) &= FV(F_1) \\ FV(\exists v.F_1) &= FV(F_1) \setminus \{v\} \\ FV(\forall v.F_1) &= FV(F_1) \setminus \{v\} \end{aligned} \quad .$$

Validity and Equivalence

Definition. Formula F is *valid*, iff for all e, $e \models F \leftrightarrow [\![F]\!]_e = 1$.

Observation: F **is valid iff** $\neg F$ **is unsatisfiable**. *Unsatisfiable means always evaluates to false. Hence, if not F is always false, then F is always true, a.k.a, valid.*

Definition. Formulas F and G are equivalent iff for every e that defines all variables in $FV(F) \cup FV(G)$, we have: $e \models F$ iff $e \models G$.

Observation: F and G are equivalent iff $F \leftrightarrow G$ is valid. *Means that for any arbitrary environment, F and G hold the same truth value.*

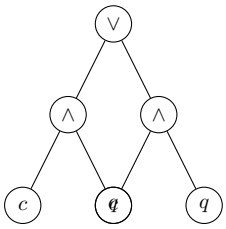
$\exists p.F$ us equivalent to $F[p := 0] \vee F[p := 1]$ whereas $\forall p.F$ to $F[p := 0] \wedge F[p := 1]$.

Boolean Function Representation: Circuits

Formulas correspond to *trees*: variables are leaves, operations internal nodes.
More efficient representation that exploits sharing: directed acyclic graphs (DAGs). We can view DAGs as formulas with *auxiliary* variable definitions.

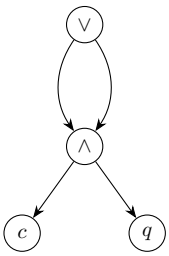
1. Syntax Tree

$F = (c \wedge q) \vee (c \wedge q)$



2. Shared DAG

Exploiting subformula sharing



3. Formula View

With auxiliary variables

Auxiliary Defs:
$T_1 = c \wedge q$
$T_2 = T_1 \vee T_1$
Top-Level Formula:
$F = T_2$

Example: simple (ripple-carry) n-bit adder:

- input numbers: $s_1, ..., s_n$ and $a_1, ..., a_n$
- output: $s'_1, ..., s'_n$

The formula with auxiliary variables $c_1, ..., c_{n+1}$:

$$c_1 = 0 \wedge \bigwedge_{i=1}^{n} (s'_i = s_i \oplus a_i \oplus c_i) \wedge (c_{i+1} = (s_1 \wedge a_i) \vee (s_i \wedge c_i) \vee (a_i \wedge c_i)).$$

We can implement such definitions in hardware: route an output of one gate to multiple other gates.
To get back a tree: substitute all auxiliary variables c_i, but we get much bigger formula. Or, existentially quantify all auxiliary variables.

Formula Representation of Sequential Circuits

We represent sequential circuit as $C = (\bar{s}, \text{Init}, R, \bar{x}, \bar{a})$ where:

- $\bar{s} = (s_1, ..., s_n)$ is the vector of state variables
- *Init* is a boolean formula with *FV(Init)* $\subseteq \{s_1, ..., s_n\}$i
- $\bar{a} = (a_1, ..., a_m)$ is the vector of input variables
- $\bar{x} = (x_1, ..., x_k)$ is the vector of auxiliary variables (for R)
- R is a boolean formula called transition formula, for which

$$\text{FV(R)} \subseteq \{s_1, ..., s_n, a_1, ..., a_m, x_1, ..., x_k, s'_1, ..., s'_n\}$$

Transition system for C is *(S, I, r, A)* where $S = \{0,1\}^n, A = \{0,1\}^m$,

- $I = \{\bar{v} \in \{0,1\}^n \mid [\bar{s} \mapsto \bar{v}] \models Init\}$
- $r = \{(\bar{v}, \bar{u}, \bar{v}') \in \{0,1\}^{n+m+n} \mid [(\bar{s}, \bar{a}, \bar{s}') \mapsto (\bar{v}, \bar{u}, \bar{v})] \models \exists \bar{x}.R\}$

Auxiliary variables $\bar{x}$ are treated as existentially quantified, can use conjucts $x_i = E(\bar{s}, \bar{a}, \bar{x})$ to express intermediate values.

What needs to be remembered: We can represent faithfully and, in many cases, efficiently, transition systems using propositional formulas for initial state and transition relation.

Checking Inductive Invariant using SAT queries

Given the sequential circuit representation $C = (\bar{s}, Init, R, \bar{x}, \bar{a})$ and a formula Inv with $FV(Inv) \subseteq \{s_1, ..., s_n\}$, how do we check that Inv is an inductive invariant?
Let us write *negations* of conditions "$Init \subseteq Inv$" and "$r[Inv] \subseteq Inv$"

- An initial state is not included in invariant:

$$Init \wedge \neg Inv$$

- There is a state satisfying invariant, leading to a state that breaks invariant:

$$\underbrace{Inv}_{\bar{s}} \wedge \underbrace{R}_{\bar{s},\bar{a},\bar{x},\bar{s}'} \wedge \underbrace{\neg Inv[\bar{s} := \bar{s}']}_{\bar{s}'}$$

  Note that $\bar{a}, \bar{x}$ variables are also existentially quantified, as they should be.

We can check if a formula is an inductive invariant using two queries to a SAT solver and making sure that they both return **unsat**.

Appendix: Scala Snack Dispenser

Description

Snack dispenser has 6 levels with 4 slots each. Each slot can hold 10 items. Each item costs 2 CHF. The only way to operate a machine is to insert 1 CHF coin into temporary storage (one step), which must be done two times, then select the refreshment (in one step), which immediately dispenses the snack or cancels and returns coins if none is available in the slot, or if the stable coin storage is full. The machine can hold up to 500 CHF in its stable coin storage. It starts full with items but with no coins. Describe this transition system and estimate the cardinality of the set of all of its states. Can the stable storage ever hold 99 coins? 100 coins? 490 coins?

states: $10^{4 \cdot 6} \cdot 501 \cdot 2 = 10^{24} \cdot 1002 > 10^{27}$

coins: $0 \to 2 \to ... \to 98 \to 100 \to 240 \cdot 2 = 480$

Code

```
import stainless.lang._
import stainless.collection._
object Dispenser {
  abstract sealed class Action
  case class FirstCoin() extends Action
  case class SecondCoin(levelNo: BigInt, slotNo: BigInt) extends Action {
    require(0 <= levelNo && levelNo <= 5 &&
            0 <= slotNo && slotNo <= 3)
  }

  case class State(
    levels: List[List[BigInt]],
    coins: BigInt,
    oneInserted: Boolean)
  {
    require(levels.size == 6 &&
            levels.forall(l => l.size == 4) &&
            0 <= coins && coins <= 500)
  }

  def isInitial(s: State): Boolean = {
    s.levels.forall(l => l.forall(itemNo => itemNo==10)) &&
    s.coins == 0 &&
    s.oneInserted == false
  }

  def r(s1: State, a: Action): Option[State] = {
    a match {
      case FirstCoin() if !s1.oneInserted && s1.coins < 500 =>
        Some(s1.copy(
          oneInserted = true,
          coins = s1.coins + 1))
      case SecondCoin(levelNo,slotNo) if s1.oneInserted && s1.coins < 500 => {
        val level: List[BigInt] = s1.levels(levelNo)
        val amount = level(slotNo)
        if (amount > 0) {
          val newLevel: List[BigInt] = level.updated(slotNo, amount - 1)
          Some(s1.copy(
            levels = s1.levels.updated(levelNo, newLevel),
```

```
            oneInserted = false,
            coins = s1.coins + 1))
      } else None()
    }
    case _ => None()
  }
}

def isTraceLike(t: List[(State,Action)]): Boolean = {
  t match {
    case Cons((s1,a1), Cons((s2,a2),rest)) => {
      if (r(s1,a1)==Some(s2)) isTraceLike((s2,a2)::rest)
      else false
    }
    case _ => true
  }
}

def isTrace(t: List[(State, Action)]): Boolean = { // last action is ignored
  t match {
    case Cons((s,a),_) => isInitial(s) && isTraceLike(t)
    case _ => false
  }
}

def exampleTrace: List[(State,Action)] = {
  val b10 = BigInt(10)
  val s0 = State(
    levels = List(
      List(b10,b10,b10,b10),
      List(b10,b10,b10,b10),
      List(b10,b10,b10,b10),
      List(b10,b10,b10,b10),
      List(b10,b10,b10,b10),
      List(b10,b10,b10,b10)),
    coins = BigInt(0),
    oneInserted=false)
  val Some(s1) = r(s0, FirstCoin())
  val Some(s2) = r(s1, SecondCoin(0,0))
  List[(State,Action)]((s0, FirstCoin()), (s1, SecondCoin(0,0)), (s2, FirstCoin
  ()))
}.ensuring(res => isTrace(res))

def prop(s: State): Boolean = {
  s.coins % 2 == (if (s.oneInserted) BigInt(1) else BigInt(0))
}

def propInductive(t: List[(State, Action)]): Unit = {
  require(isTraceLike(t) &&
    (t match {
      case Cons((s,a),_) => prop(s)
      case _ => false
    })
  )
  t match {
    case Cons((s,a),Nil()) => ()
    case Cons((s,a),rest) => {
      assert(rest.last == t.last)
      propInductive(rest)
```

```
98        }
99      }
100   }.ensuring(_ => prop(t.last._1))
101
102   def propInvariant(t: List[(State, Action)]): Unit = {
103     require(isTrace(t))
104     propInductive(t)
105   }.ensuring(_ => prop(t.last._1))
106 }
```